

Invenqor Server 설치·운영·오프라인 배포 가이드

대상 Server 버전: v0.2.7 · Agent 버전: v0.2.7 · 기준일: 2026-07-29

1. 운영 구조와 단일 포트

Invenqor는 Linux Agent, 중앙 Server, PostgreSQL, 웹 관리 콘솔로 구성됩니다. 사용자 UI, 관리자 API, Agent 수집·heartbeat·업데이트 트래픽은 모두 Server의 기본 TCP 7070 한 포트를 공유합니다. 방화벽에는 외부 공개 포트를 여러 개 추가하지 않아도 됩니다.



운영망에서는 외부 HTTPS 443의 Ingress 또는 Reverse Proxy에서 TLS를 종료하고 내부 Service 7070으로 전달합니다. Agent의 `server.url`은 `https://invenqor.example.com`처럼 외부 URL을 지정하며 외부가 기본 443이면 7070을 붙이지 않습니다. Agent는 inbound 포트를 열지 않고 Server로 outbound 연결만 생성합니다.

2. 무엇을 수집하고 어떻게 저장하는가

영역	주요 항목	기본 원천
시스템	호스트명, 배포판, Kernel, Architecture, Boot time, Timezone	<code>/etc/os-release</code> , <code>/proc</code>
CPU·메모리	모델, 논리 CPU, load, 메모리·swap 지표	<code>/proc/cpuinfo</code> , <code>/proc/meminfo</code>
파일시스템	장치, Mount, 유형, 옵션, 용량, inode	<code>/proc/self/mounts</code> , <code>statvfs</code>
네트워크	Interface, MAC/IP, MTU, 상태, Route, DNS, 로컬 포트	<code>/sys/class/net</code> , <code>/proc/net</code>
프로세스	PID/PPID, 이름, 상태, UID/GID, 실행 파일	<code>/proc/[pid]</code>
패키지	dpkg/rpm/apk 이름, 버전, Architecture	배포판 패키지 DB 또는 고정 <code>rpm -qa</code>
서비스	systemd/OpenRC/SysV 상태와 시작 정책	각 init 조회 인터페이스
계정	사용자·그룹, UID/GID, 홈, Shell, 보조 그룹	<code>/etc/passwd</code> , <code>/etc/group</code>
컨테이너	Runtime socket, cgroup, 컨테이너 내부 여부	Runtime·cgroup 표식

프로세스 명령행은 비밀정보가 들어갈 수 있어 기본 미수집입니다. 파일 본문, 비밀번호 해시, 환경변수, 키·토큰, 패킷 내용은 수집하지 않습니다.

Server는 `Agent UUID + category + asset_id` 를 원천 키로 보관합니다. 첫 전송은 전체 Snapshot, 이후에는 `added/updated/removed` 변경분만 전송합니다. Collector 오류가 발생한 주기에는 누락을 삭제로 오판하지 않습니다. 원본 이벤트, 현재 Snapshot, 변경 이력과 오류를 함께 보존해 화면 값의 출처를 추적할 수 있습니다.

3. 온라인 Docker Compose 설치

필수 조건은 Docker Engine 24+, Compose v2, 4 GiB 이상의 여유 메모리입니다.

```
git clone https://github.com/hkjang/invenqor.git
cd invenqor
export POSTGRES_PASSWORD="$(openssl rand -base64 32)"
export BOOTSTRAP_ADMIN="admin"
export BOOTSTRAP_ADMIN_PASSWORD="CorrectHorse!42"
docker compose up -d --build
curl -fsS http://127.0.0.1:7070/health/ready
```

`http://서버:7070` 에서 콘솔을 엽니다. 운영 전에는 TLS Proxy, DNS, PostgreSQL 백업, 시간 동기화와 로그 수집을 구성하십시오.

4. 폐쇄망·오프라인 설치

GitHub Release의 두 파일을 인터넷 연결 구간에서 내려받아 승인된 매체로 반입합니다.

- `invenqor-0.2.7.tar.gz`
- `invenqor-0.2.7.tar.gz.sha256`
- 함께 제공되는 `compose.offline.yaml`

무결성 검증 후 Docker에 Server와 PostgreSQL 이미지를 한 번에 적재합니다.

```
sha256sum -c invenqor-0.2.7.tar.gz.sha256
gzip -t invenqor-0.2.7.tar.gz
docker load < invenqor-0.2.7.tar.gz
docker image inspect invenqor-server:0.2.7 --format '{{.Id}} {{.Architecture}}'
docker image inspect postgres:17-alpine --format '{{.Id}} {{.Architecture}}'
```

`compose.offline.yaml` 은 `pull_policy: never` 이므로 외부 Registry를 조회하지 않습니다.

```
export POSTGRES_PASSWORD="$(openssl rand -base64 32)"
export BOOTSTRAP_ADMIN="admin"
export BOOTSTRAP_ADMIN_PASSWORD="CorrectHorse!42"
docker compose -f compose.offline.yaml up -d
curl -fsS http://127.0.0.1:7070/health/ready
```

반입 이미지는 `linux/amd64` 용입니다. ARM 서버에는 x86_64 이미지를 실행하지 마십시오. 제공된 빌드 스크립트는 이미지 두 개를 `docker save` 호환 gzip으로 만들고 SHA-256 파일까지 생성합니다.

```
./scripts/build-offline-images.sh 0.2.7
```

5. 최초 관리자와 Agent 등록

셸이 없는 Distroless Server image에서는 환경변수로 초기 관리자를 자동 생성하는 방식을 권장합니다. 다음 이름은 모두 지원하지만 `INVENQOR_` 접두사가 있는 이름을 표준으로 사용하십시오.

```
docker run -d --name invenqor-server \
  -p 7070:7070 \
  -v invenqor-server-state:/var/lib/invenqor-server \
  -e INVENQOR_BOOTSTRAP_ADMIN=admin \
  -e INVENQOR_BOOTSTRAP_ADMIN_PASSWORD='CorrectHorse!42' \
  invenqor-server:0.2.7
```

Compose는 호스트의 `BOOTSTRAP_ADMIN` 과 `BOOTSTRAP_ADMIN_PASSWORD` 를 위 표준 환경변수로 전달합니다. 소문자 `bootstrap_admin` , `bootstrap_admin_password` 도 직접 `docker run -e` 로 전달할 수 있습니다.

용도	표준 이름	호환 이름
관리자 ID	<code>INVENQOR_BOOTSTRAP_ADMIN</code>	<code>BOOTSTRAP_ADMIN</code> , <code>bootstrap_admin</code>
관리자 비밀번호	<code>INVENQOR_BOOTSTRAP_ADMIN_PASSWORD</code>	<code>BOOTSTRAP_ADMIN_PASSWORD</code> , <code>bootstrap_admin_password</code>
비밀번호 파일	<code>INVENQOR_BOOTSTRAP_ADMIN_PASSWORD_FILE</code>	<code>BOOTSTRAP_ADMIN_PASSWORD_FILE</code> , <code>bootstrap_admin_password_file</code>

비밀번호 값과 비밀번호 파일을 동시에 지정하면 안전을 위해 Server가 기동을 거부합니다. 비밀번호 파일 경로는 컨테이너 내부의 절대 경로여야 하며 마지막 개행 문자는 제거해서 사용합니다.

이 값은 **사용자가 한 명도 없는 최초 DB에서만** Super Admin을 생성합니다. 재시작하거나 여러 Pod가 동시에 기동해도 기존 계정과 비밀번호는 변경하지 않으며, 성공 후 일회성 Token과 DB claim을 폐기합니다. 비밀번호는 정책 검증 후 Argon2id hash로만 저장되고 로그·감사 내역에는 기록되지 않습니다.

환경변수 값은 `docker inspect` 또는 일부 운영 도구에서 보일 수 있으므로 계정 생성 후 Compose 환경에서 비밀번호를 제거하십시오. Kubernetes에서는 `INVENQOR_BOOTSTRAP_ADMIN_PASSWORD_FILE` 과 Secret volume 방식을 사용합니다.

환경변수를 사용하지 않은 경우에는 기존 일회용 Token API도 유지됩니다. Token 파일은 상태 볼륨을 마운트한 임시 진단 컨테이너에서 읽을 수 있습니다.

```
docker run --rm --volumes-from invenqor-server:ro \
  --entrypoint /bin/sh postgres:17-alpine \
  -c 'cat /var/lib/invenqor-server/initial-admin.token'
```

```
curl -X POST http://127.0.0.1:7070/api/v1/bootstrap/admin \
  -H "X-Invenqor-Bootstrap-Token: $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"CorrectHorse!42","display_name":"관리자"}'
```

성공 즉시 토큰은 DB에서 폐기되어 재사용할 수 없습니다.

개별 Agent UUID 등록과 장비별 Token 복사는 기본 절차가 아닙니다. 기본값은 URL-only 자동 등록입니다. Agent `config.toml` 에 Server URL만 입력하면 최초 연결에서 로컬 device claim을 만들고 장비별 Bearer Token을 발급받습니다.

```
[server]
url = "https://invenqor.example.com:7070"
ca_file = "/etc/invenqor-agent/ca.pem"
allow_insecure_http = false
timeout_seconds = 30
```

Agent는 로컬 claim과 서버가 발급한 `ivq_at_...` 장비 토큰을 `/var/lib/invenqor-agent/{enrollment-claim,device-credential}.json` 에 0600 으로 보존합니다. 응답 유실 또는 장비 토큰 무효화 시 같은 claim으로 자동 복구하며, 다른 서버 URL에는 기존 장비 토큰을 보내지 않습니다.

URL-only 모드는 Server 7070에 도달할 수 있는 장비의 최초 등록을 허용합니다. 인터넷 또는 신뢰하지 않는 네트워크에 노출하는 경우 32자 이상의 공용 Token을 설정하면 같은 자동화 흐름을 유지하면서 등록 요청을 보호할 수 있습니다.

```
export AGENT_ENROLLMENT_TOKEN="ivq_et_$(openssl rand -hex 32)"
docker compose up -d
```

Kubernetes에서는 모든 Server Pod가 같은 Secret을 읽어야 합니다. `INVENQOR_AGENT_ENROLLMENT_TOKEN_FILE` 또는 Helm `agentEnrollmentTokenSecret.name/key` 를 사용하고 Agent에도 같은 Token 파일을 배포합니다. 자동 등록 자체를 금지하려면 `INVENQOR_AGENT_AUTO_ENROLLMENT=false` 또는 Helm `agentAutoEnrollment: false` 를 사용합니다.

위 환경변수는 공용 DB에 등록 정책이 아직 없을 때의 **최초 기동 기본값**입니다. 최초 정책 생성 뒤에는 **설정 → Agent 등록** 화면의 DB 정책이 모든 Server Pod에 우선하며, 정책 변경에 재기동이나 sticky session이 필요하지 않습니다.

자동 등록을 사용하지 않는 예외 장비는 관리 화면에서 UUID를 수동 등록하고 한 번 표시되는 `bearer_token` 을 구성할 수 있습니다. 서버 DB에는 어느 방식이든 장비 Token 원문이 아닌 SHA-256 해시만 저장됩니다.

localhost, RFC1918/사설 IP, 단일-label 내부 DNS와 `.internal` / `.local` 주소의 HTTP는 URL-only 설치를 위해 자동 허용합니다. 공인 DNS의 HTTP는 `allow_insecure_http=true` 를 명시해야 하지만 격리된 E2E망 외에는 사용하지 마십시오. 신뢰 경계를 넘는 운영 연결은 HTTPS를 유지하십시오.

6. Agent 설치와 실제 기동

Release의 CPU별 정적 musl 패키지를 사용합니다. 이 방식은 CentOS 7처럼 오래된 glibc가 있는 호스트에도 별도 런타임을 요구하지 않습니다.

```
curl -fLO https://github.com/hkjang/invenqor-agents/releases/download/v0.2.7/invenqor-agent-linux-x86_64.tar.gz
curl -fLO https://github.com/hkjang/invenqor-agents/releases/download/v0.2.7/invenqor-agent-linux-x86_64.tar.gz.sha256
sha256sum -c invenqor-agent-linux-x86_64.tar.gz.sha256
tar -xzf invenqor-agent-linux-x86_64.tar.gz
sudo ./invenqor-agent-linux-x86_64/scripts/install.sh
sudoedit /etc/invenqor-agent/config.toml
sudo /opt/invenqor-agent/bin/invenqor-agent --validate-config
sudo systemctl restart invenqor-agent
sudo systemctl status invenqor-agent --no-pager
```

SysV와 OpenRC 정의도 패키지에 포함됩니다. 상태 디렉터리와 `agent-id` 는 재설치·업데이트 중 보존해야 같은 장비로 계속 인식됩니다.

7. 관리 부담을 줄이는 자동 동작

- Server는 시작 시 DB Schema를 자동 마이그레이션합니다.
- PostgreSQL 멀티 파드 마이그레이션은 advisory lock으로 한 Pod씩 수행됩니다.
- 일시적 DB 장애에는 인증된 Agent 이벤트를 Pod별 append-only spool에 쓰고 DB 복구 뒤 자동 재처리합니다.
- Agent는 변경분만 보내고, 전송 실패 시 내구성 큐와 지수 backoff로 자동 재시도합니다.
- Agent는 최초 연결에서 자동 등록하며, 장비 Token 무효화 시 로컬 claim으로 자동 재등록합니다. 관리 API의 차단은 즉시 모든 Pod에 적용됩니다.
- 서명된 업데이트는 Agent가 주기적으로 확인·검증·스테이징하고 systemd path unit이 root helper를 한 번만 호출해 원자 교체합니다.

사람이 반드시 수행할 작업은 초기 관리자, TLS/서명 개인키 보관, 백업 복구 훈련과 업데이트 승인입니다. 외부 노출 환경에서는 enrollment token의 안전한 배포·회전도 포함합니다. 개별 Agent 등록과 장비별 Token 복사는 필요하지 않습니다.

8. 서명된 Agent 자동 업데이트

8.1 키 준비

업데이트 전용 Ed25519 개인키는 Server에 두지 말고 오프라인 서명 환경에 보관합니다. 같은 공개키를 Agent(검증용)와 Server(게시 시점 검증용)에 각각 설정합니다.

```
openssl genpkey -algorithm ED25519 -out update-private.pem
openssl pkey -in update-private.pem -pubout -outform DER |
  tail -c 32 | base64 | tr -d '\n'          # 이 값을 양쪽에 설정
openssl pkeyutl -sign -rawin -inkey update-private.pem \
  -in invenqor-agent -out invenqor-agent.sig # 이 .sig 파일을 그대로 업로드
```

Server:

```
INVENQOR_UPDATE_PUBLIC_KEY="위에서-구한-base64-공개키"
# 또는 INVENQOR_UPDATE_PUBLIC_KEY_FILE=/run/secrets/update-public-key
```

Agent:

```
[updates]
enabled = true
channel = "stable"
check_interval_seconds = 21600
public_key = "위에서-구한-base64-공개키"
install_path = "/opt/invenqor-agent/bin/invenqor-agent"
```

Server 공개키를 반드시 설정하십시오. 설정하지 않으면 Server는 서명 형식만 확인합니다. 잘못된 서명으로 게시해도 게시는 성공하고, 그 뒤 fleet의 모든 Agent가 검증에 실패합니다. 증상은 각 호스트 로그 한 줄뿐이므로 발견이 늦습니다. 공개키가 있으면 게시 시점에 `UPDATE_SIGNATURE_REJECTED` 로 거부되어 잘못을 즉시 알 수 있습니다. 콘솔 게시 화면에도 현재 검증 가능 여부가 표시됩니다.

8.2 게시와 단계적 확대

Agent 관리 화면에서 artifact와 `.sig` 파일을 올리고 버전·아키텍처·최초 rollout을 입력하면 게시됩니다. `.sig` 는 `openssl pkeyutl -sign` 이 만든 64 byte 원본을 그대로 올려도 되고, base64 문자열을 붙여넣어도 됩니다(줄바꿈 허용).

게시 후에는 **재업로드 없이** rollout을 조절합니다.

작업	API	콘솔
릴리즈·적용 현황 조회	<code>GET /api/v1/admin/agent-updates</code>	게시된 릴리즈 목록

작업	API	콘솔
rollout 확대	PATCH /api/v1/admin/agent-updates/{release}	10 / 25 / 50 / 100% 버튼
즉시 배포 중단	같은 API에 rollout_percent: 0	중단 버튼
릴리즈 삭제	DELETE /api/v1/admin/agent-updates/{release}	휴지통 버튼

권장 절차는 10% → 확인 → 25% → 50% → 100%입니다. 화면의 진한 막대는 해당 버전을 이미 보고한 Agent 비율, 옅은 막대는 현재 rollout 대상 비율입니다. 문제가 보이면 **중단**을 누르십시오. 즉시 어떤 Agent도 그 릴리즈를 제안받지 않습니다.

Rollout 대상 선정은 Agent UUID 해시로 결정되므로 같은 호스트가 항상 같은 순번에 들어갑니다. Canary가 매번 다른 장비로 바뀌면 비교가 불가능하기 때문입니다. 20,000대 시뮬레이션에서 각 구간 편차는 ± 10% 이내입니다.

8.3 롤백

이미 적용된 Agent를 되돌리려면 **이전 버전을 롤백 릴리즈로 게시**합니다. `allow_downgrade`가 설정된 릴리즈만 Agent가 하위 버전으로 받아들이며, 서명과 SHA-256 검증은 동일하게 수행합니다. 이 표시가 없으면 Agent는 자신보다 낮은 버전을 거부하므로, 잘못된 릴리즈에 갇힌 fleet을 꺼낼 수 없습니다.

8.4 적용 경로와 안전장치

Agent는 비특권 계정으로 실행되므로 스스로 바이너리를 교체하지 않습니다. 검증이 끝난 업데이트는 `updates/pending.json`으로 스테이징되고, 실제 설치 권한을 가진 경로가 수행합니다.

init	적용 시점
systemd	<code>invenqor-agent-update.path</code> 가 스테이징을 감지해 즉시 적용하고 서비스를 재시작
OpenRC	서비스 시작 시 <code>start_pre</code> 에서 적용
SysV	서비스 시작 시 적용

설치 직전에 스테이징된 바이너리를 실제로 실행해 `--version`을 확인합니다. 서명과 해시가 맞아도 실행되지 않는 빌드(아키텍처 계열 불일치, 잘못된 빌드)를 활성화하면 그 릴리즈를 받은 모든 호스트에서 수집이 멈추고 장비마다 손으로 복구해야 합니다. 자기 점검에 실패하면 설치를 중단하고 기존 바이너리를 그대로 유지합니다. 성공 시 기존 바이너리는 `.previous`로 남고 rename 실패 시 즉시 복원되며, 스테이징 파일은 적용 후 삭제됩니다.

관리자가 직접 즉시 갱신할 때는 한 번의 명령으로 끝냅니다.

```
sudo /opt/invenqor-agent/bin/invenqor-agent \
  --config /etc/invenqor-agent/config.toml --update-now
```

확인, 다운로드, 서명·해시 검증, 자기 점검, 설치를 순서대로 수행하고 결과를 출력합니다. 설치 권한이 없으면 스테이징까지만 진행하고 종료 코드 3과 함께 필요한 명령을 알려 주므로, 운영 중 Agent는 영향을 받지 않습니다.

현재 상태는 Agent에서 바로 확인할 수 있습니다.

```

invenqor-agent --config /etc/invenqor-agent/config.toml --status
# updates      자동 · 실행 0.2.7 · 대기 0.2.8
invenqor-agent --config /etc/invenqor-agent/config.toml --diagnose
# [WARN] automatic updates a verified update is staged and is waiting to be installed

```

개인이기가 유출되면 즉시 모든 릴리즈를 중단하고 공개키를 교체한 뒤 새 패키지를 배포하십시오.

9. Kubernetes 멀티 파드

필수 조건은 외부 PostgreSQL, 모든 Pod가 공유하는 32-byte Master Key Secret, Pod별 RWO state PVC, 업데이트용 RWX PVC입니다.

```

head -c 32 /dev/urandom > master.key
kubectl create secret generic invenqor-master-key --from-file=master.key
kubectl create secret generic invenqor-database \
  --from-literal=dsn='postgres://user:password@host/db?sslmode=require'
kubectl create secret generic invenqor-bootstrap-admin \
  --from-literal=password='CorrectHorse!42'
helm upgrade --install invenqor deploy/helm/invenqor \
  --set replicaCount=2 \
  --set bootstrapAdmin.username=admin \
  --set bootstrapAdmin.passwordSecret.name=invenqor-bootstrap-admin \
  --set updates.storageClassName='YOUR-RWX-STORAGE-CLASS'

```

Chart는 StatefulSet parallel 기동, readiness/liveness/startup probe, Pod anti-affinity, rolling update와 PDB `minAvailable: 1` 을 포함합니다. 세션, RBAC, Agent 상태와 감사 로그는 PostgreSQL에 있으므로 어느 Pod로 접속해도 동일합니다. Master Key가 Pod마다 다르면 암호화된 OIDC/TOTP 비밀을 해독할 수 없으므로 Secret을 교체하거나 분실하지 마십시오. RWX StorageClass가 없는 클러스터는 업데이트 공유 저장소를 별도 Object Storage 구현으로 대체하기 전까지 replica 1로 운영해야 합니다.

Chart는 관리자 비밀번호 Secret을 `/run/secrets/invenqor-bootstrap/password` 에 읽기 전용으로 마운트하고 `INVENQOR_BOOTSTRAP_ADMIN_PASSWORD_FILE` 로 읽습니다. 초기 계정 생성이 확인되면 Helm values에서 `bootstrapAdmin.username` 을 비우고 해당 Secret을 폐기하십시오.

9.1 HTTPS Ingress

Chart의 선택적 Ingress는 `/` Prefix 하나로 웹, 관리 API, Agent 등록·이벤트·업데이트를 모두 같은 origin에 전달합니다. NGINX Ingress 예시는 다음과 같습니다.


```

ingress:
  enabled: true
  className: nginx
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: 20m
    nginx.ingress.kubernetes.io/proxy-read-timeout: "90"
  hosts:
    - host: invenqor.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: invenqor-tls
      hosts: [invenqor.example.com]

```

Ingress는 경로를 rewrite하지 않아야 하며 Agent 이벤트 본문 상한 16 MiB보다 큰 20 MiB 이상을 허용해야 합니다. Agent는 다음 경로를 같은 HTTPS origin으로 사용합니다.

목적	경로
최초 자동 등록	POST /v1/agent/enroll
인벤토리·heartbeat	POST /v1/agent/events
업데이트 확인·다운로드	GET /v1/agent/updates...

사실 인증서이면 각 Agent의 `ca_file` 에 CA chain을 배포합니다. IP allowlist를 사용하면 설정 → Agent 등록 → 신뢰 프록시에 Ingress의 실제 Pod/노드 IP 또는 CIDR을 추가하고, Ingress Controller가 수신한 임의 `X-Forwarded-For` 를 신뢰하지 않고 표준 방식으로 덮어쓰도록 구성합니다. Server는 TCP peer가 신뢰 프록시 목록에 있을 때만 이 헤더를 사용합니다.

10. 상태 확인, 백업과 복구

경로	정상 기준
/health/live	HTTP 200, 프로세스 생존
/health/ready	HTTP 200, 요청 처리 준비
/health/database	POSTGRES_ACTIVE 권장
/api/v1/system/info	버전 0.2.7, 포트 7070, DB 모드

백업 대상은 PostgreSQL, Pod별 state/spool PVC, 업데이트 RWX PVC와 Master Key Secret입니다. DB와 Master Key는 같은 복구 시점으로 보호하십시오. 복구 훈련은 별도 Namespace에서 로그인, Agent 재전송, 감사 로그와

update manifest까지 확인해야 완료입니다.

11. 검증된 호환성

v0.2.7 E2E는 실제 PostgreSQL-backed Server와 Agent 컨테이너를 기동하고 수집 레코드 생성, 인증 전송, DB 처리, daemon 지속 실행과 서명 업데이트 스테이징을 확인했습니다.

OS 이미지	결과
Alpine	통과
CentOS 7	통과
Red Hat UBI 8	통과
Red Hat UBI 9	통과
Ubuntu 22.04 LTS	통과
Ubuntu 24.04 LTS	통과

별도 E2E에서 Server 두 Pod를 동시에 시작해 migration, readiness, 교차 Pod 로그인 세션을 확인했습니다. 재현 명령은 `./scripts/e2e-client-server.sh` 와 `./scripts/e2e-multipod.sh` 입니다.

12. 장애 판단

12.1 등록이 안 될 때 확인 경로

등록 실패는 Agent가 Server에 도달했는지에 따라 확인 지점이 다릅니다. 아래 세 경로 중 하나에는 반드시 기록이 남습니다.

상황	확인 위치	내용
Server에 도달함	콘솔 Agent 관리 → 등록 진단	최근 24시간 등록 성공·거부 건수, 출처 IP별 마지막 원인 코드와 조치, 등록 후 첫 수집이 없는 Agent
Server에 도달함	콘솔 Server 진단 로그	<code>request_id</code> , 처리 Pod, 판정 IP, 정책 버전 포함 원본 이벤트
Server에 도달 못함	Agent 장비	<code>invenqor-agent --diagnose</code> , <code>--status</code> , <code>/var/lib/invenqor-agent/status.json</code>

Agent 없이 등록 가능 여부만 확인하려면 사전 점검 API를 호출합니다. 자격 증명 없이 호출할 수 있고, Server가 인식한 출처 IP와 거부 사유를 그대로 돌려줍니다. Authorization 헤더에 장비 Token을 함께 보내면 그 Token의 유효성까지 판정합니다.

```
curl -s https://invenqor.example.com:7070/v1/agent/preflight | jq .
```

```
{
  "observed_source_ip": "10.20.7.31",
  "enrollment": {
    "mode": "open", "network_mode": "allowlist", "network_allowed": false,
    "would_enroll": false, "reason": "AGENT_SOURCE_NOT_ALLOWED"
  },
  "credential": {"presented": false, "state": "absent"}
}
```

이 호출은 상태를 만들지 않지만 진단 로그에는 `AGENT_PREFLIGHT_READY` 또는 `AGENT_PREFLIGHT_BLOCKED` 로 남으므로, 등록에 실패한 장비의 시도도 콘솔에서 확인할 수 있습니다. 잘못된 경로로 들어온 Agent 요청은 `AGENT_ENDPOINT_NOT_FOUND` 로 기록되며 HTML 대신 JSON 404를 반환합니다.

관리 API로 요약을 직접 조회할 수도 있습니다(`agents.read` 권한).

```
curl -s -b cookies "$BASE/api/v1/admin/diagnostics/enrollment?hours=24" | jq .totals
```

12.2 증상별 판단

- Agent 로그의 `code=...` `request_id=...` 를 **Server 로그** 화면에서 검색하면 요청 처리 Pod, source IP, 정책 버전과 안전하게 정리된 내부 원인을 확인할 수 있습니다. 모든 응답에 `X-Request-Id` 헤더가 포함되므로 프록시 로그에서도 같은 ID로 대조할 수 있습니다.
- Agent가 목록에 아예 없음: 등록 자체가 실패한 것입니다. 12.1의 **등록 진단** 패널에서 출처 IP별 원인 코드를 확인하고, 기록이 없다면 Agent가 Server에 도달하지 못한 것이므로 해당 장비에서 `--diagnose` 를 실행합니다.
- Agent는 있으나 자산이 없음: 등록은 됐고 수집 이벤트가 없는 상태입니다. **등록 진단**의 첫 수집 대기 목록과 Agent의 `--status` 큐 깊이를 함께 확인합니다.
- `401` : Agent UUID와 장비별 Token, 차단 여부를 확인합니다.
- `403 AGENT_SOURCE_NOT_ALLOWED` : IP/CIDR allowlist, 신뢰 프록시와 `X-Forwarded-For` 판정을 확인합니다.
- `5xx` : Agent의 request ID로 Server 진단 로그를 찾고, 같은 시각의 해당 Pod stdout과 PostgreSQL 상태를 함께 확인합니다.
- TLS 오류: URL hostname, 사설 CA, 인증서 만료와 7070 경로를 확인합니다.
- 큐 증가: Server readiness와 네트워크를 복구하십시오. 큐를 임의 삭제하지 않습니다.
- `SQLITE_FALLBACK` : PostgreSQL DSN/DNS/TLS/인증을 수정하고 운영 전 PostgreSQL 모드로 재기동합니다.
- 업데이트 미적용: 공개키, signature, SHA-256, version, architecture, rollout bucket과 systemd update path unit을 확인합니다.
- 멀티 파드 일부 실패: 공통 Master Key, RWX 권한, PostgreSQL advisory lock, 해당 Pod의 state/spool PVC를 확인합니다.

13. Agent 자동 등록 설정

`settings.read` 권한은 현재 정책을 조회하고, `settings.write` 권한은 **설정** → **Agent 등록**에서 다음 세 모드를 즉시 전환할 수 있습니다.

모드	신규 Agent 동작	권장 용도
토큰 없이 자동 등록 (<code>open</code>)	<code>config.toml</code> 의 <code>server.url</code> 만으로 최초 통신 시 등록	접근이 통제된 사내망의 Zero-touch 배포
등록 토큰 필요 (<code>token</code>)	URL과 공용 등록 Token이 모두 맞아야 최초 등록	외부 또는 신뢰 경계가 넓은 네트워크
자동 등록 비활성 (<code>disabled</code>)	신규 등록만 HTTP 403으로 거부	동결 기간·침해 대응·폐쇄 운영

Open 모드의 최소 Agent 설정은 아래와 같습니다. 사전 자산 생성, 장비별 Token 복사나 Agent 재시작 전의 별도 승인 절차는 필요하지 않습니다.

```
[server]
url = "https://invenqor.example.com:7070"
```

토큰 발급은 256-bit 등록 Token을 만들고 즉시 Token 보호 모드로 전환합니다. 같은 버튼을 다시 누르면 회전되며 구 Token은 즉시 무효화됩니다. 원문은 발급 응답과 화면에 한 번만 표시되고 DB·감사 로그에는 저장되지 않습니다. **토큰 폐기** 후 자동 등록이 활성 상태라면 Open 모드가 됩니다. 이 동작은 기존 Agent가 보유한 장비별 `ivq_at_...` Token이나 정상 수집에는 영향을 주지 않습니다.

정책은 공용 PostgreSQL의 `server_metadata` 에 버전과 함께 저장됩니다. 각 등록 요청이 DB의 현재 값을 검증하므로 여러 Server Pod가 동시에 실행되어도 변경 직후 동일한 결과를 냅니다. 모든 활성화·비활성화·발급·회전·폐기는 변경 사유와 전후 상태를 감사 로그에 남기며 Token 원문과 해시는 노출하지 않습니다.

IP/CIDR 기반 Zero-touch 등록

설정 → **Agent 등록** → **자동 등록 허용 IP / CIDR**에서 등록 요청의 출발지를 추가로 제한할 수 있습니다.

- **모든 IP 허용**은 기존처럼 인증 모드만 적용합니다.
- **지정 IP만 허용**은 단일 IPv4/IPv6 또는 CIDR과 일치할 때만 등록합니다.
- 허용 규칙은 최대 256개이며 공백·중복과 CIDR host bit는 저장 시 정규화됩니다.
- 허용되지 않은 요청은 자격 증명을 만들기 전에 `AGENT_SOURCE_NOT_ALLOWED` 로 거부됩니다.

```
10.20.30.40
10.20.40.0/24
2001:db8:100::/64
```

Kubernetes Ingress나 L7 Load Balancer 뒤에서는 **신뢰 프록시 IP / CIDR**에 실제 Ingress/LB의 접속 주소만 등록하십시오. Server는 TCP peer가 이 목록과 일치할 때만 `X-Forwarded-For` 를 오른쪽부터 검증해 실제 Agent IP를 판정합니다. 목록 밖의 클라이언트가 전달 헤더를 위조해도 직접 접속 IP로 판정됩니다. 프록시 Pod CIDR 전체를 넣기보다 가능한 한 전용 Ingress node/Pod 대역을 사용하십시오.

허용된 Agent의 `/v1/agent/enroll` 이 성공하면 Server는 인벤토리 전송을 기다리지 않고 다음 레코드를 같은 DB 트랜잭션에서 만듭니다.

- `status=discovered` , `type=host` 인 임시 자산
- 접속 IP의 `asset_identifiers(type=ip)` 식별자
- Agent UUID와 접속 IP가 포함된 `enrollment` source

첫 `system` 인벤토리가 도착하면 별도 host를 만들지 않고 이 자산을 `active` 로 승격하고 `hostname·OS·architecture` 등의 authoritative payload를 병합합니다. 따라서 관리 화면에는 등록 직후 자산이 보이고, 수집 완료 뒤에도 중복 host가 남지 않습니다. 이 기능은 능동 포트 스캔이 아니라 실제 Agent 접속 기반 등록이므로 방화벽에는 기존 단일 TCP `7070` 만 필요합니다.

동일 기능의 관리 API는 다음과 같습니다. 상태 변경에는 관리자 Session, `X-CSRF-Token` , `settings.write` 가 필요합니다.

Method	경로	기능
GET	<code>/api/v1/admin/settings/agent-enrollment</code>	현재 정책과 Token 설정 여부
PATCH	<code>/api/v1/admin/settings/agent-enrollment</code>	인증 모드와 IP/CIDR·신뢰 프록시 정책 적용
POST	<code>/api/v1/admin/settings/agent-enrollment/token</code>	Token 발급 또는 즉시 회전
DELETE	<code>/api/v1/admin/settings/agent-enrollment/token</code>	Token 폐기

14. PostgreSQL 설정 화면과 환경변수

Super Admin은 설정 → PostgreSQL에서 현재 DB 모드, 대상 host/port/database, schema, 설정 원천과 최근 기동 실패를 비밀번호 없이 확인할 수 있습니다. 새 DSN은 다음 순서로 처리됩니다.

1. 연결 테스트는 일회성 connect/ping만 수행하며 schema를 변경하거나 값을 저장하지 않습니다.
2. 검증 후 저장은 연결 성공 후 DSN을 `bootstrap.enc` 에 AES-256-GCM으로 암호화합니다. API와 화면에는 비밀번호나 원문 DSN을 다시 반환하지 않습니다.
3. Server를 순차 재기동하면 저장값이 적용됩니다. SQLite 데이터는 자동 이관하지 않으므로 운영 데이터가 있다면 별도 백업·이관 승인이 필요합니다.
4. Kubernetes에서는 화면의 Pod 로컬 저장값 대신 모든 Pod에 동일한 Secret 환경변수를 배포하고 rolling restart 합니다.

환경변수 우선순위는 아래와 같습니다. 앞의 값이 있으면 뒤의 값과 화면 저장값을 덮어씁니다.

우선순위	이름	용도
1	<code>INVENQOR_POSTGRES_DSN</code>	권장 표준 이름
2	<code>POSTGRES_DSN</code>	Compose 호환 이름
3	<code>postgres_dsn</code>	기존 배포 호환용 소문자 이름

우선순위	이름	용도
4	암호화된 화면 저장값	단일 인스턴스 편의 구성

```
docker run -d --name invenqor-server \
  -p 7070:7070 \
  -e postgres_dsn='postgres://invenqor:password@db:5432/invenqor?sslmode=require' \
  -v invenqor-server-state:/var/lib/invenqor-server \
  invenqor-server:0.2.7
```

환경변수가 적용 중이면 화면에 **환경변수 우선** 경고가 표시됩니다. 이때 화면에서 다른 DSN을 저장해도 실행 중인 값은 바뀌지 않으며, 해당 환경변수를 제거하고 재기동해야 암호화 저장값이 적용됩니다.

15. Keycloak SSO/OIDC 구성

Keycloak에서는 Invenqor용 **confidential OpenID Connect client**를 만들고 Standard Flow를 활성화합니다. Direct Access Grant와 Implicit Flow는 끕니다.

Keycloak 항목	권장값
Valid Redirect URI	<code>https://invenqor.example.com/api/v1/auth/keycloak/callback</code>
Valid Post Logout Redirect URI	서비스 기본 URL
Web Origin	서비스 기본 URL
Client authentication	On
PKCE	S256

Invenqor의 설정 → Keycloak → 최소 정보 빠른 연동에는 다음 값만 입력합니다.

1. Keycloak 주소(예: `https://sso.example.com`)
2. Realm
3. Client ID와 최초 Client Secret
4. Invenqor 외부 주소(현재 브라우저 origin이 기본값)

자동 구성 · 검증 · 활성화는 Realm issuer를 조합하고 OIDC Discovery와 TLS/사설 CA를 검증한 뒤 Callback URI, Logout URI, `openid profile email`, 표준 사용자 claim과 Keycloak 기본 `realm_access.roles` claim을 구성합니다. 검증에 실패하면 설정과 새 Secret을 저장하지 않습니다. 이미 Secret이 저장된 경우에는 Secret을 다시 입력하지 않고 URL/Realm/Client ID만 재검증할 수 있습니다. 생성된 Callback URI는 성공 메시지와 고급 설정에서 확인하여 Keycloak Client의 Valid Redirect URI와 일치시키십시오.

고급 정책에서는 Redirect/Logout URI, Scope, 사용자·Email·이름·역할·그룹 claim, 허용 Email domain, 기본 역할과 자동 사용자 생성을 세밀하게 조정합니다. 사내 TLS를 쓰면 빠른 연동 전에 루트/중간 CA 인증서를 PEM 입력란에

붙여 넣습니다.

역할과 그룹 매핑은 한 줄에 하나씩 입력합니다.

```
# Keycloak realm role = Invenqor role
inventory-admin=asset_manager
inventory-read=viewer

# Keycloak full group path = Invenqor role
/invenqor/operators=operator
/invenqor/auditors=auditor
```

Keycloak 표준 realm role을 직접 읽으려면 Role Claim에 `realm_access.roles` 를 지정할 수 있습니다. 점(.)으로 구분한 중첩 claim을 지원합니다. 별도 protocol mapper로 평면 `roles` claim을 ID Token에 넣어도 됩니다. 그룹은 Group Membership mapper에서 **Full group path**와 ID Token 포함을 활성화하십시오.

저장 전 확인 사항:

- Issuer는 HTTPS이고 discovery endpoint에 Server가 접근할 수 있어야 합니다.
- Scope에는 반드시 `openid` 가 포함되어야 합니다.
- 활성화 시 Client Secret이 필수이며 암호화 저장되고 다시 노출되지 않습니다.
- 기본 역할과 모든 mapping 대상은 실제 Invenqor 역할이어야 합니다.
- 허용 Email domain을 지정했다면 Email Claim도 필수입니다.
- **연결 테스트**는 Issuer discovery와 TLS/사설 CA 신뢰를 검증합니다.

최초 로그인은 Authorization Code + PKCE S256, State, Nonce를 검증합니다. 기존 SSO 사용자는 로그인할 때마다 프로필과 `keycloak` 원천 역할이 동기화되며, Keycloak에서 회수된 역할도 제거됩니다. 관리 콘솔에서 비활성화하거나 삭제한 계정은 같은 Keycloak subject로 자동 재생성되지 않습니다. 로컬 로그인도 비상 접근 경로로 유지하되 최소 한 개의 로컬 Super Admin을 별도로 보호하십시오. Logout Redirect URI가 설정된 SSO 사용자는 로컬 Session 폐기 후 Keycloak end-session endpoint로 이동해 IdP Session 종료도 이어서 수행합니다.

15.1 SSO 로그인 실패 확인

Keycloak 로그인도 브라우저 이동으로 진행되므로 실패해도 화면에 JSON이 표시되지 않습니다. Server는 로그인 화면으로 되돌려보내며 원인 코드와 request ID를 함께 표시하고, 같은 내용을 **Server 진단 로그**의 `keycloak` 구성요소에 조치 문구와 함께 기록합니다.

코드	원인	조치
<code>KEYCLOAK_DISABLED</code>	자동 로그인 비활성	설정 → Keycloak에서 활성화
<code>KEYCLOAK_SECRET_REQUIRED</code>	Client Secret 미설정	Secret을 입력하고 저장
<code>KEYCLOAK_UNREACHABLE</code>	discovery 실패	URL·realm·DNS·TLS 신뢰 확인, 연결 테스트 실행
<code>KEYCLOAK_PROVIDER_REJECTED</code>	Keycloak이 요청 거부	client의 Valid Redirect URI와 인증 흐름 정책 확인

코드	원인	조치
KEYCLOAK_FLOW_EXPIRED	10분 초과 또는 재사용	로그인을 다시 시작
KEYCLOAK_USERNAME_CONFLICT	같은 이름의 로컬 계정 존재	로컬 계정을 정리하거나 다른 username claim 사용
KEYCLOAK_EMAIL_DOMAIN_REJECTED	허용 도메인 밖	도메인 허용 목록 수정
KEYCLOAK_PROVISIONING_DISABLED	자동 생성 비활성	계정을 먼저 생성하거나 자동 생성 활성화
KEYCLOAK_USER_INACTIVE	연결 계정 비활성	사용자 화면에서 활성화
KEYCLOAK_ROLE_MISSING	매핑이 없는 역할을 지정	역할 매핑 수정

Client Secret이 없는 상태로 활성화되어 있으면 로그인 화면은 Keycloak 버튼을 표시하지 않고 미완성 상태임을 안내합니다. 동작하지 않는 버튼을 노출하지 않기 위한 동작입니다.

Session Cookie와 HTTPS: Session Cookie는 요청이 HTTPS일 때만 **Secure** 로 발급됩니다. TLS를 상위 Proxy에서 종료하는 경우 **X-Forwarded-Proto: https** 를 전달하도록 구성하십시오. 평문 HTTP로 운영하면 Cookie에 **Secure** 가 붙지 않으며, 신뢰 경계를 넘는 트래픽은 반드시 HTTPS를 사용해야 합니다.

16. 자산 분류와 관계 자동화

수집 결과는 그대로 두면 업무 맥락이 없습니다. 환경은 전부 **other**, 중요도는 전부 **normal** 이고 관계는 손으로 입력해야 합니다. **설정 → 자산 분류**는 이 두 가지를 자동화하면서 근거를 남깁니다.

16.1 분류 규칙

규칙은 우선순위 순서로 실행되고, 뒤의 규칙은 앞의 규칙이 정한 값을 조건으로 쓸 수 있습니다. 별도 엔진 없이 “운영 환경의 데이터베이스는 치명”을 표현하기 위한 구조입니다.

단계	우선순위	하는 일
수집 범주 정규화	10	system → host, service → service, software.package → software 등 표준 유형 부여
소프트웨어 역할	40	알려진 DB·웹·컨테이너·메시지 계층을 승격하고 태그를 붙이며 호스트 관계 생성
환경 추론	60	호스트 이름 토큰으로 production/staging/qa/test/development 판정
중요도 추론	80~84	환경과 유형 조합으로 치명·높음·낮음 판정

환경 규칙은 부분 문자열이 아니라 구분자로 나눈 **토큰**을 비교합니다. ***stg*** 같은 부분 문자열 규칙은 **postgresql** 에도 걸리기 때문입니다. 또한 환경은 호스트에서만 판정하고, 그 호스트의 구성요소는 호스트의 환경을 물려받습니다. 구성요소의 환경은 그것이 올라간 장비의 환경이라는 것이 사실에 가깝습니다.

각 자산에는 적용된 규칙 목록, 확신도, 분류 시각이 함께 저장됩니다. 확신도는 값을 정한 규칙 중 **가장 낮은 값을 씁니다**. 약한 추측 하나가 확정된 사실처럼 보이지 않게 하기 위한 선택입니다.

운영자 지정 우선: 콘솔이나 API로 사람이 유형·환경·중요도·담당부서· 위치를 수정하면 해당 필드는 그 사람의 것이 되고 이후 자동 실행이 되돌리지 않습니다. 어떤 필드가 사람 소유인지는 자산의 `manual_fields` 에 기록됩니다.

규칙을 켜고 끄거나 우선순위를 바꾼 뒤 **저장된 자산 재분류**를 실행하면 이미 수집된 자산에 즉시 반영됩니다. 앞으로 수집될 자산만 바뀌는 분류는 쓸모가 없습니다.

16.2 관계 추론과 검토 대기열

Agent 한 번의 수집 결과에 들어 있는 레코드는 모두 같은 장비에서 읽은 것입니다. 이 동일 장비 사실만으로 다음 관계를 확신도와 근거(`same_agent_inventory`)와 함께 만듭니다.

관계	대상
<code>part_of</code>	네트워크 인터페이스, CPU·메모리, 네트워크 구성
<code>attached_to</code>	마운트된 파일시스템 볼륨
<code>runs_on</code>	컨테이너 런타임, 그리고 역할 규칙이 붙은 DB·웹·메시지 서비스

모든 레코드에 관계를 만들지는 않습니다. 리눅스 호스트 한 대의 설치 패키지는 수천 건이라, 패키지마다 관계를 만들면 아무도 읽을 수 없는 그래프가 됩니다. 어떤 구성요소가 관계를 가질지는 규칙의 `relate_to_host` 로 결정하는 큐레이션 사항입니다. 실측: 자산 1,434건에서 관계는 196건입니다.

추론하지 않는 것도 의도된 설계입니다.

- **서비스 간 의존 관계:** Agent는 수신 소켓만 보고하고 상대방은 보고하지 않으므로 “A가 B에 의존” 은 만들어낸 값이 됩니다. 변경관리가 그 값을 근거로 판단하게 되므로, 없는 것이 잘못된 것보다 낫습니다. 상대방 연결을 수집하는 collector가 추가되면 구현합니다.
- **자동 병합:** 서로 다른 Agent가 같은 machine identifier를 보고하면 대개 이미지 복제입니다. 이는 `duplicate_of` 제안으로만 남기고 병합 여부는 사람이 결정합니다. 잘못 병합한 호스트를 되돌리는 비용이 크기 때문입니다.

제안은 **설정** → **자산 분류** 하단 검토 대기열에 나타납니다. 승인하면 사람이 결정한 관계(`source>manual`)가 되어 이후 자동 실행이 바뀌지 않고, 거부하면 상태만 `rejected` 로 남아 같은 제안이 다시 올라오지 않습니다.

관리 API:

```
GET    /api/v1/admin/settings/classification      # settings.read
PATCH /api/v1/admin/settings/classification/rules/{id} # settings.write
POST   /api/v1/admin/settings/classification/reclassify # settings.write
GET    /api/v1/assets/relations/proposed          # relations.read
POST   /api/v1/assets/relations/{id}/approve|reject # relations.write
```

17. 사용자 관리

사용자 화면은 계정 생성, 검색, 프로필 수정, 활성화/비활성, 잠금 해제, 로컬 비밀번호 초기화, 역할 부여와 삭제를 제공합니다.

- 비밀번호 변경 시 Argon2id로 다시 hash하고 기존 세션을 모두 폐기합니다.
- 비활성화와 삭제는 사용자 Session과 해당 사용자가 만든 API key를 즉시 폐기합니다.
- 현재 로그인한 관리자는 자기 계정을 비활성화·강등·삭제할 수 없습니다.
- 마지막 활성 Super Admin은 비활성화·강등·삭제할 수 없습니다.
- SSO 표시 역할은 매 로그인마다 Keycloak에서 동기화되므로 콘솔에서 제거할 수 없습니다. 별도 로컬 역할만 콘솔에서 추가·회수할 수 있습니다.
- Keycloak 사용자의 비밀번호와 프로필 원본은 Keycloak에서 관리합니다.
- 권한은 역할에서만 나오므로 마지막 역할은 회수할 수 없습니다. 접근을 막아야 하면 계정을 비활성화하십시오. Keycloak이 부여한 역할이 남아 있는 계정은 로컬 역할을 모두 회수할 수 있습니다.
- 삭제한 계정의 사용자 ID는 즉시 재사용할 수 있습니다. 삭제된 행은 목록에서 숨기고 이름을 원래이름#deleted-
<id> 형태로 보존하므로 감사 추적은 유지되며 같은 ID로 새 계정을 만들 수 있습니다.
- 모든 관리 변경은 행위자, 대상, 결과와 사유를 감사 로그에 기록합니다.